CS 331 — Software Engineering Lab

Assignment 7: Business Logic Layer

Project: AI-Powered Natural Language Linux Terminal

Submitted by:

Rishabh Kumar Jain (2301175)
Rahul Yadav (2301168)
Prakshay Saini (2301149)

Computer Science and Engineering
Indian Institute of Information Technology, Guwahati

Academic Year: 2025–2026

Question 1: Business Logic Layer — Core Functional Modules

The Business Logic Layer (BLL) of the AI-Powered Natural Language Linux Terminal is the central nervous system of the application. It sits between the Presentation Layer (the terminal-style GUI) and the Infrastructure Layer (the OS shell executor), mediating every command that flows through the system.

1.1 Overview of the BLL and Its Role

In our Layered Architecture, the BLL is responsible for:

- **Interpreting user intent:** Receiving raw natural language text from the UI and routing it to the AI model for processing.
- **Enforcing safety:** Validating every AI-generated command through a rule-based safety filter before it is allowed to reach the shell.

- **Coordinating execution:** Passing validated, safe commands down to the execution layer and returning results back up to the UI.
- **Providing context:** Maintaining session history so the LLM can generate contextually relevant, sequential commands.

1.2 Core Functional Modules of the Business Logic Layer

There are three primary modules that constitute the BLL of this project:

Module 1: CommandController

The CommandController is the central orchestrator of the entire application. It is the first component to receive input from the TerminalUI (Presentation Layer) and the last to send output back to it. It acts as the traffic director — deciding whether a given input should be treated as a standard direct shell command or as a natural language instruction requiring LLM processing.

Key Responsibilities:

- Receives raw string input from the TerminalUI.
- Detects whether the input is a natural language query or a direct shell command.
- Calls the LLMAPIConnector for NL inputs, receiving a generated shell command.
- Passes the generated command to the SafetyValidator before any execution.
- Calls the CommandExecutor for safe, validated commands.
- Returns the final output (or a safety-block message) back to the TerminalUI.
- Updates the SessionHistoryManager with each interaction for contextual follow-up.

Pseudocode Implementation:

```
class CommandController:
    def handle_input(self, user_input):
        if self.is_natural_language(user_input):
            history = self.session_history.get_recent()
            shell_cmd = self.llm_connector.translate(user_input, history)
        else:
            shell_cmd = user_input

        if self.safety_validator.is_safe(shell_cmd):
            output = self.executor.run(shell_cmd)
```

```
        else:
            output = '[BLOCKED] Command flagged as unsafe.'

        self.session_history.log(user_input, shell_cmd, output)
        return output
```

Module 2: LLMAPIConnector

The LLMAPIConnector is responsible for all communication with the external AI (Large Language Model) API. It encapsulates the complexity of prompt engineering, API request formatting, and response parsing. The UI and the CommandController never need to know how the AI model works internally — they simply ask the connector to translate a natural language query and receive a shell command string in return.

Key Responsibilities:

- Constructs a structured prompt using the user's natural language input and recent session history to provide contextual awareness.
- Makes a secure, authenticated API call to the LLM provider (e.g., Gemini, OpenAI, or a local model).
- Parses the AI's raw text response to cleanly extract only the executable shell command, stripping any explanatory prose.
- Handles API failures gracefully — network timeouts, malformed responses, or quota errors — and returns an appropriate error message to the controller without crashing the session.

Pseudocode Implementation:

```
class LLMAPIConnector:
    def translate(self, user_input, history):
        system_prompt = self.build_system_prompt()
        context = self.format_history(history)
        full_prompt = f'{context}\nUser says: {user_input}'

        try:
            response = api_client.complete(system_prompt, full_prompt)
            return self.extract_command(response.text)
        except APIError as e:
            return None  # Controller handles None gracefully
```

The system prompt injected into every API call constrains the LLM to respond only with valid Linux shell commands, avoiding ambiguous or conversational responses. For example:

```
System: You are a Linux command generator. Given a natural language instruction,
output ONLY the single equivalent Linux shell command. Do not explain.
Do not include markdown. Output ONLY the raw command.
```

Module 3: SafetyValidator

The SafetyValidator is the security gatekeeper of the system. Because LLM outputs are non-deterministic — the AI may occasionally generate incorrect or destructive commands — this module ensures that no harmful shell instruction ever reaches the OS. It operates entirely within the BLL, acting as a mandatory checkpoint between LLM output and command execution.

Key Responsibilities:

- Maintains a rule-set of forbidden command patterns and dangerous flags.
- Parses and analyzes each generated command using string matching and pattern recognition.
- Returns a boolean safe/unsafe verdict to the CommandController.
- Logs all blocked commands for audit and debugging purposes.

Blocked Command Examples:

- rm -rf / or rm -rf /* — recursive deletion from root
- :(){ :|:& };: — fork bomb
- dd if=/dev/zero of=/dev/sda — disk wipe
- chmod -R 777 / — permission escalation on root
- shutdown, reboot, halt, poweroff — system state commands
- mkfs.* — filesystem formatting commands

Pseudocode Implementation:

```
class SafetyValidator:
    BLOCKED_PATTERNS = [
        r'rm\s+-rf\s+[/~]', r'dd\s+if=',
        r'mkfs', r':(\(\)\s*\{)',
        r'shutdown', r'reboot', r'halt'
```

```
    ]


    def is_safe(self, command):
        for pattern in self.BLOCKED_PATTERNS:
            if re.search(pattern, command, re.IGNORECASE):
                self.log_blocked(command)
                return False
        return True
```

1.3 Interaction Between BLL and the Presentation Layer (TerminalUI)

The Presentation Layer (TerminalUI) interacts with the BLL exclusively through the CommandController interface. The UI has no direct access to the LLM, the safety rules, or the shell executor — this is the core benefit of the layered architecture.

Step-by-step interaction flow:

- User types a natural language command (e.g., 'show me all PDF files in the documents folder') in the TerminalUI text input field and presses Enter.
- The TerminalUI captures the keystroke event and calls controller.handle_input(user_input). This is the sole interaction point between the Presentation and Business Logic layers.
- The CommandController detects the natural language intent, calls the LLMAPIConnector, and receives back: find ~/Documents -name '*.pdf'
- The CommandController passes the generated command to SafetyValidator.is_safe(). The command is clean, so the validator returns True.
- The CommandController calls the CommandExecutor in the Infrastructure Layer. The shell runs the command and returns the list of PDF files.
- The output string is returned to the TerminalUI, which renders it in the output panel alongside the original NL input and the generated command, so the user can learn from it.

Presentation Layer	Business Logic Layer	Infrastructure Layer
TerminalUI Captures user input Renders output	CommandController LLMAPIConnector SafetyValidator SessionHistoryManager	CommandExecutor OS Shell Interface subprocess module
Interaction: Calls controller.handle_input() → receives output string	*Interaction: Calls executor.run() → receives shell output*	*Returns: stdout, stderr, exit code to BLL*

Question 2: Business Rules, Validation Logic, and Data Transformation

Part A: Business Rules Implementation

Business rules are the core policies and conditions that govern how a software system operates — they define what is allowed, what is blocked, and how the system should behave in each scenario. In the AI-Powered Natural Language Linux Terminal, business rules are implemented across all three primary BLL modules.

Business Rule 1: Natural Language vs. Direct Command Detection

The system must decide whether a given input is a standard Linux command (e.g., ls -la) or a natural language instruction (e.g., 'list all files in the current directory sorted by size'). This is a fundamental business rule that governs routing logic in the CommandController.

Implementation: The CommandController applies a heuristic detection function that checks for the absence of a known command prefix and the presence of English-language patterns (spaces, articles, verbs, question words). If detected as NL, input is routed to the LLM; otherwise, it is passed directly through validation to execution.

Rule in code:

```
def is_natural_language(self, text):
    KNOWN_COMMANDS = ['ls', 'cd', 'mkdir', 'grep', 'find', 'cat', 'echo',
...]
    first_word = text.strip().split()[0].lower()
    return first_word not in KNOWN_COMMANDS
```

Business Rule 2: Command Safety Policy

The system enforces a strict 'deny by default' policy for dangerous commands. Every command — whether entered directly or generated by the LLM — must pass through the SafetyValidator before execution. This is the most critical business rule in the entire application.

Implementation approach: The SafetyValidator uses a multi-layered approach to enforce safety rules:

- **Pattern Matching:** Uses regular expressions to detect destructive patterns like rm -rf /, dd if=/dev/zero, or fork bombs.
- **Keyword Blacklist:** Maintains a list of banned commands such as shutdown, reboot, mkfs, format.
- **Flag Analysis:** Inspects command flags — for example, -rf with rm targeting root or home is blocked even if rm itself is permitted.
- **Privilege Escalation Check:** Commands beginning with sudo or su are blocked by policy to ensure execution remains within normal user-level permissions.

The business rule can be articulated as: 'A command is safe if and only if it matches no blocked pattern, targets no restricted path, and requires no elevated privileges.'

Business Rule 3: LLM Response Acceptance Policy

The system must validate that the LLM's response is actually a shell command and not conversational text, an explanation, or markdown. The business rule is: 'Only accept LLM responses that are single-line, start with a valid Unix command token, and contain no natural language prose.'

Implementation:

```
def extract_command(self, raw_response):
    lines = raw_response.strip().splitlines()
    # Remove markdown code fences if present
    cmd = lines[0].strip().lstrip('`').rstrip('`')
    if self.looks_like_command(cmd):
        return cmd
    return None  # Signals that LLM output was invalid
```

Business Rule 4: Session Context Window Limit

The SessionHistoryManager enforces a business rule that limits how many past interactions are sent to the LLM as context. Sending the entire session history would increase API token usage and latency. The rule is: 'Provide only the last N interactions (typically 5) as context for the LLM to ensure relevant, low-latency responses.'

```
def get_recent(self, n=5):
```

```
    return self.history[-n:]  # Sliding window of last 5 interactions
```

Business Rule 5: Command Recommendation Engine

After a command is successfully executed, the system applies a rule to suggest a contextually relevant follow-up command. The business rule is: 'After executing a command of type X, recommend a command of type Y.' This is implemented as a rule table mapping actions to suggestions.

Executed Command Type	Recommendation	Example
mkdir (create folder)	Open / navigate to folder	`cd <folder_name>`
touch (create file)	Edit or view the file	`nano <filename>`
find (search files)	Open containing folder	`ls <directory>`

Part B: Validation Logic

Validation logic ensures that data entering the system is correct, consistent, and in the proper format before it is processed further. In the context of this application, data flows through several transformation stages, and validation is applied at each critical point.

Validation Point 1: User Input Validation (Presentation → BLL Boundary)

Before the TerminalUI passes user input to the CommandController, basic input validation occurs at the boundary:

- **Empty Input Check:** If the user presses Enter without typing anything, the system does not make an API call. The UI simply re-displays the prompt. This prevents unnecessary LLM API costs and empty session log entries.
- **Whitespace Normalization:** Leading and trailing whitespace is stripped from the input string. Multiple internal spaces are collapsed to single spaces to prevent shell command parsing errors.
- **Encoding Validation:** Non-UTF-8 characters are detected and either sanitized or rejected with an error message to prevent shell injection through encoding tricks.

Code:

```
def validate_input(self, raw_text):
    if not raw_text or not raw_text.strip():
```

```
        return None, 'Input cannot be empty.'
    sanitized = ' '.join(raw_text.strip().split())
    try:
        sanitized.encode('utf-8')
    except UnicodeEncodeError:
        return None, 'Invalid characters detected.'
    return sanitized, None
```

Validation Point 2: LLM Response Validation (LLMAPIConnector)

The output from the external LLM API is inherently unpredictable. The LLMAPIConnector applies strict validation to ensure the extracted command is usable:

- **Non-empty Response Check:** The API response must not be empty or whitespace-only. An empty response triggers a fallback error message rather than attempting execution.
- **Single-command Validation:** The system expects a single shell command, not a shell script or multi-line response. If the LLM returns multiple commands (e.g., a pipeline with newlines), only the first logical command is accepted, or the entire response is rejected if it appears to be a script.
- **Markdown Stripping:** LLMs often wrap commands in markdown code fences (```bash ... ```). The connector validates and strips these before returning the command. If the content inside the fence is empty, it returns None.
- **Command Token Validation:** The first token of the extracted command must be a recognizable Unix binary name. If the first word is something like 'Sure' or 'Here', it indicates the LLM returned prose instead of a command, and the result is rejected.

Code:

```
def extract_command(self, raw_response):
    if not raw_response or not raw_response.strip():
        return None
    text = raw_response.strip()
    # Strip markdown fences
    if text.startswith('```'):
        lines = text.splitlines()
        text = '\n'.join(lines[1:-1]).strip()
    first_line = text.splitlines()[0].strip()
    # Reject if first word is an English prose word
    prose_starters = ['sure', 'here', 'the', 'to', 'you', 'this']
```

```
    if first_line.split()[0].lower() in prose_starters:
        return None
    return first_line
```

Validation Point 3: Safety Validation (SafetyValidator)

This is the most critical validation step. The SafetyValidator acts as the final gatekeeper between the BLL and the Infrastructure Layer:

- **Pattern-Based Regex Validation:** Commands are matched against a library of dangerous patterns using compiled regular expressions for performance. Each pattern is designed to catch both exact matches and common variations of dangerous commands.
- **Path-Based Validation:** Commands that target root paths (/), system directories (/etc, /sys, /proc, /boot), or home directory shortcuts (~) with destructive operations are flagged and blocked.
- **Flag Combination Validation:** The validator checks for dangerous combinations of flags. For example, rm with -r and -f together is blocked when targeting root or home, even if rm alone would be permitted.
- **Privilege Escalation Check:** Any command prefixed with sudo or su is automatically blocked, ensuring all operations remain within standard user permissions as defined by the security requirements.

Code:

```
import re


class SafetyValidator:
    BLOCKED_PATTERNS = [
        r'\brm\s+.*-[rR][fF].*[/~]',
        r'\bdd\b.*if=/dev/(zero|null|random)',
        r'\bmkfs\b', r'\bformat\b',
        r':\(\)\s*\{.*\|.*&',  # fork bomb
        r'\b(shutdown|reboot|halt|poweroff)\b',
        r'\bsudo\b', r'\bsu\b',
        r'chmod\s+-R\s+777\s+/',
    ]


    def is_safe(self, command):
```

```
        for pattern in self.BLOCKED_PATTERNS:
            if re.search(pattern, command, re.IGNORECASE):
                return False
        return True
```

Validation Point 4: Execution Output Validation (CommandExecutor → BLL)

After a command executes, the CommandExecutor returns the output along with an exit code. The BLL validates this return to determine how to present it to the user:

- **Exit Code Interpretation:** An exit code of 0 means success; any non-zero exit code is interpreted as an error, and the system displays the stderr output to the user with a clear error prefix, distinguishing it from normal output.
- **Output Length Limit:** If the command produces an extremely large output (e.g., cat of a multi-megabyte file), the BLL truncates it to a configurable maximum number of lines (e.g., 1000 lines) and informs the user that output was truncated, preventing the UI from becoming unresponsive.

```
def format_output(self, stdout, stderr, exit_code):
    if exit_code != 0:
        return f'[Error (exit {exit_code})]: {stderr.strip()}'
    lines = stdout.splitlines()
    if len(lines) > 1000:
        return '\n'.join(lines[:1000]) + '\n... [output truncated]'
    return stdout
```

Part C: Data Transformation

Data transformation refers to the process of converting data from the format in which it originates — in this project, from the shell and the LLM API — into a format suitable for the presentation layer (the TerminalUI). This is a critical concern in our application because data passes through multiple representation changes on its journey from the shell to the user's screen.

Transformation 1: Natural Language → Structured Prompt (User Input → LLM)

When a user types a natural language instruction, the raw string must be transformed into a structured prompt before it can be sent to the LLM API. This is not a trivial conversion — a poorly structured prompt yields poor commands.

Transformation Steps:

- **Step 1 — Context Injection:** The SessionHistoryManager provides the last 5 interactions. These are formatted as a conversation history block and prepended to the prompt. This transforms isolated context-free text into a contextual, multi-turn dialogue.
- **Step 2 — System Role Framing:** A system-level instruction is prepended that constrains the LLM's behavior. The user's input is wrapped in a role-tagged message structure required by the API.
- **Step 3 — API Request Serialization:** The structured prompt is serialized into the JSON format expected by the API endpoint, including model name, max_tokens, and message array.

Example transformation:

```
# Raw user input:
'show all running processes using more than 100mb of memory'


# Transformed API request:
{
  "model": "gemini-pro",
  "messages": [
    {"role": "system", "content": "Output only valid Linux commands..."},
    {"role": "user", "content": "[history]\nUser: show all running ..."}
  ]
}
```

Transformation 2: LLM Raw Text Response → Clean Shell Command String

The LLM API returns a JSON response containing a text field. This raw text undergoes several transformations before it becomes a usable shell command:

- **JSON Deserialization:** The API response is parsed from JSON into a Python dictionary, and the relevant text field is extracted.
- **Markdown Stripping:** Code fences (```bash, ```) are stripped. The LLMAPIConnector extracts only the command content from within the fences.
- **Whitespace Normalization:** Trailing newlines, carriage returns, and extra spaces are stripped to produce a clean, executable command string.

- **Comment Removal:** Inline comments (# ...) that some LLMs append as explanations are removed, since these would cause the shell to print unexpected output.

Before and after transformation example:

```
# LLM Raw Response (before transformation):
```bash
ps aux --sort=-%mem | awk 'NR>1 && $6 > 102400'  # shows >100MB
```


After transformation (clean, executable command):
ps aux --sort=-%mem | awk 'NR>1 && $6 > 102400'
```

Transformation 3: Shell Execution Output → Formatted UI Display String

This is the most complex transformation in the application, converting raw bytes and mixed encoding from the OS shell into a human-readable, UI-renderable string.

- **Byte Decoding:** The subprocess module returns stdout and stderr as bytes. These are decoded using UTF-8, with an 'errors=replace' fallback to handle files with mixed or invalid encoding, replacing undecodable bytes with the Unicode replacement character (?) instead of crashing.
- **ANSI Escape Code Handling:** Many terminal commands (like ls --color, grep --color) produce ANSI color escape codes (e.g., \x1b[32m for green). The TerminalUI must either parse these into corresponding text colors or strip them, depending on whether the UI supports rich text rendering.
- **Line Ending Normalization:** Shell output may use \r\n (CRLF), \n (LF), or \r (CR) depending on the command and locale. All line endings are normalized to \n for consistent rendering in the UI text widget.
- **Output Annotation:** The BLL wraps the output string with metadata: the original NL input, the generated command (shown in a different color/style), and the output. This annotation transforms a raw shell string into a structured, educational display that shows the user both what was run and what it produced.
- **Error Differentiation:** Stdout and stderr are kept separate by the executor and recombined by the BLL only after labeling them. Errors are prefixed with [stderr] or a red-colored indicator, transforming the undifferentiated shell output into a clearly structured success/failure display.

Complete transformation chain example:

```
# Infrastructure Layer returns:
```

```
stdout = b'\x1b[32mfile1.pdf\x1b[0m\nfile2.pdf\n'
stderr = b''
exit_code = 0

# BLL Transformation:
stdout_str = stdout.decode('utf-8', errors='replace')
clean_output = strip_ansi(stdout_str)  # 'file1.pdf\nfile2.pdf\n'
normalized = clean_output.replace('\r\n', '\n').replace('\r', '\n')

# Final UI Display String (annotated):
display = f'> {user_nl_input}\n'
       + f'$ {generated_command}\n'
       + f'{normalized}'
```

Transformation 4: Session Interaction Log → LLM Context Format

The SessionHistoryManager stores raw interaction records, but these must be transformed into the specific conversational format required by the LLM API before each new request. This ensures the LLM understands previous interactions as a conversation, not as disconnected log entries.

The transformation converts a list of Python dictionaries (internal log format) into a list of role-tagged message objects (LLM API format):

```
# Internal session log (Data Layer format):
[
  { 'nl': 'list files', 'cmd': 'ls -la', 'output': 'total 48...' },
  { 'nl': 'show disk usage', 'cmd': 'df -h', 'output': 'Filesystem...' }
]

# Transformed LLM API format (Presentation-ready for the AI):
[
  {"role": "user", "content": "list files"},
  {"role": "assistant", "content": "ls -la"},
  {"role": "user", "content": "show disk usage"},
  {"role": "assistant", "content": "df -h"}
]
```

This transformation is critical — without it, the LLM would have no awareness of what the user has already asked or done, making contextual follow-up commands impossible.

— End of Assignment 7 —